RadialMenu Asset

This asset has been created by SYNERGY IT company, we develop many amazing asset's created in Uniy3d - with guarantee of perfect quality and technical support off all our products.
RadialMenu can be used in many various ways for example to create a complex interface of your game, or as an easy way to facilitate configuration in your running presentation. This kind of Gui is very intuitive, simple to use and friendly interface,

Types of menu:

Radial Menu:
Displays buttons around the the center.
Center button to move back in menu tree.

List Menu:
Displays buttons in line.
Shows active level of menu and each previous.

Normal Menu:
Displays buttons in line.
Shows only active level of menu, use Back button to move back in menu tree.

Use in Unity3d editor:

To use the assetu simply create an empty object and add a Menu component to it, or use ready prefab (Menu) attached to the project. Now just set the parameters in editor inspector and menu is ready to use.

Menu settings:

Menu Type: - sets type of menu
Round - animated, radial menu.
List - non-animated menu, displays only one active level
Normal - non-animated menu, displays also all previous levels
Path XML - path to XML file, must me located inside "Assets\Resources\"
Size - size of single button
Position - position of menu on the screen
Movable - activates the ability to navigate menu using a proper button
Min Angle - minimum angle menu can decompose
Max Angle - maximum angle menu can decomposed
Radius - distance from the middle of menu
Skin - GUI Skin object containing button's graphics
Parent Image - name of element in GUI Skin cotaining graphics of middle button

Adding styles:

To add own styles to menu create GUI Skin object. Then click your GUI Skin object and select Custom Styles. Change size of a table as needed, change name of the element to button name used in XML file and add proper texture. Create textures <name>_on and <name>_off if node is toggle.

Input XML file specification:

Sample file:

```
<menu>
        <input name="Color" type="Button">
                <input name="Red" type="Button" value="Func1"></input>
                <input name="Oragne" type="Button" value="Func2"></input>
                <input name="Blue" type="Button" value="Func3"></input>
        </input>
        <input  name="Size" type="Button">
                <input name="Small" type="Button" value="Func4"></input>
                <input name="Big" type="Button" value="Func4"></input>
        </input>
        <input name="DrawMode" type="Toggle"></input>
</menu>
```

Key labels:

<menu></menu> - only one label of this type, at the beginning and end of file

<input></input> - tag indicating a new object, in the middle we can put more of the objects of this type, this way we are able to create tree of any sizes.

Available parameters:

name - name of a particular node, as the value can take any string, however we must use the same name which will be used in the GUI Skin to describe this object

type - it takes values: ***"Button"*** **-** which is just a classic button i ***"Toggle"*** - type of field which takes vales **true** or **false**

value **-** takes any value, which we can use later in your code. For example, it could take the name of the method to be executed by pressing. You can memorize some more information here, by using special characters. ***"Object1:Method2:10"***

Adding new parameters and types:

Adding parameters and types is possible, just ad the name of the parameter (or button type) at the right place.

<input name="DrawMode" type="NewType" newParam="NewValue"></input>

Editing scripts:

NodeType.cs - is an implementation of the standard node in terms of graphs. It has a basic, useful function specific to this data structure. The class is ready to expand, new functionality can be easily used as a new menu options. If you need a new field in node object, it would be also useful to create methods to support access to variable values. If you want to add a new type of button ensure to add it in enum: NodeType.

Menu.cs - main class of the project, includes method used to load menu form XMLfile and also managing different types of menu display, and menu animation.

1. You can add your own types of menu, or edit an existing menu display. For example, changing the position values in the draw methods (NormalMenu, ListMenu, RoundMenu).

2. If you edit the XML file structure you must also change the function used to load a file (CreateTreeFromXML).

   In case of adding new parameters remember to handle them in GetXMLNode method. (SetNewParam is your own method created to handle new parameter's values)

   ```
   newNode.SetNewParam(XMLnode.GetValue("@NewParam"));
   ```

   If you added a new button type in XML file, you will also need to add code that will support this type in CreateTreeFromXML method. (Also, remember to handle the button clicks of this type in the method OnNodeClick)

   ```
   if(XMLnode.GetValue("@type") == "TypeName")
       newNode.SetNodeType(NodeType.TypeName);
   ```

3. Handle new types of buttons in the method OnNodeClick. Add at the end of method:

   ```
   else if(node.Type == NodeType.Toggle)
   {
       //  Depending on the need handle clicking on the appropriate way.
       Debug.Log(node.GetName()+": "+node.ToggleTrigger);
   }
   ```